Простое голосование классификаторов

Обучающая выборка: $X^\ell = (x_i, y_i)_{i=1}^\ell$, $x_i \in X$, $y_i \in \{-1, +1\}$
Базовые классификаторы: $b_1(x), \ldots, b_T(x)$, $b_t \colon X \to \{-1, +1\}$

Простое голосование базовых классификаторов:

$$a(x) = \operatorname{sign} \sum_{t=1}^{T} b_t(x)$$

Композиция $a(x)$ может быть лучше базовых $b_1(x), \ldots, b_T(x)$, если они лучше случайного гадания и достаточно различны.

Способы повышения различности базовых классификаторов:

- обучение по случайным подвыборкам,
- обучение по выборке со случайными весами объектов,
- обучение по случайным подмножествам признаков,
- использование различных моделей классификации,
- использование различных начальных приближений,
- использование рандомизации при обучении $b_1, \ldots, b_T$.

Бэггинг и метод случайных подпространств

Бэггинг (bagging, bootstrap aggregating) [Breiman, 1996]:
$b_t(x)$ обучаются независимо по случайным подвыборкам длины ℓ с повторениями (как в методе bootstrap),
доля объектов, попадающих в выборку: $(1 - \frac{1}{e}) \approx 0.632$

Метод случайных подпространств
(RSM, random subspace method) [Ho, 1998]:
$b_t(x)$ обучаются по случайным подмножествам n' признаков.

Совместим обе идеи в одном алгоритме.

$\mathscr{F} = \{f_1, \ldots, f_n\}$ — признаки,
$\mu(\mathscr{G}, U)$ — метод обучения алгоритма по подвыборке $U \subseteq X^\ell$, использующий только признаки из $\mathscr{G} \subseteq \mathscr{F}$.

Breiman L. Bagging predictors // Machine Learning, 1996.

Ho T. K. The random subspace method for constructing decision forests // IEEE Trans. on Pattern Analysis and Machine Intelligence, 1998.

Бэггинг и метод случайных подпространств

Вход: обучающая выборка X^ℓ; параметры: T

ℓ' — длина обучающих подвыборок;

n' — длина признакового подописания;

ε_1 — порог качества базовых алгоритмов на обучении;

ε_2 — порог качества базовых алгоритмов на контроле;

Выход: базовые алгоритмы b_t, $t = 1, \ldots, T$;

1: **для всех** $t = 1, \ldots, T$
2: $\quad U_t :=$ случайное подмножество X^ℓ длины ℓ';
3: $\quad \mathscr{G}_t :=$ случайное подмножество $\mathscr{F}$ длины n';
4: $\quad b_t := \mu(\mathscr{G}_t, U_t)$;
5: $\quad$ **если** $Q(b_t, U_t) > \varepsilon_1$ или $Q(b_t, X^\ell \setminus U_t) > \varepsilon_2$ **то**
6: $\quad\quad$ не включать b_t в композицию;

Композиция — простое голосование: $a(x) = \operatorname{sign} \sum\limits_{t=1}^{T} b_t(x)$.

Случайный лес (Random Forest)

Обучение случайного леса:

- бэггинг над решающими деревьями
- усечение дерева (pruning) не производится
- признак в каждой вершине дерева выбирается из случайного подмножества k из n признаков
- для регрессии рекомендуется $k = \lfloor n/3 \rfloor$
- для классификации рекомендуется $k = \lfloor \sqrt{n} \rfloor$

Подбор числа деревьев T по критерию *out-of-bag*: число ошибок на объектах x_i, если не учитывать голоса деревьев, для которых x_i был обучающим:

$$\text{out-of-bag}(a) = \sum_{i=1}^{\ell} \left[\text{sign}\Big(\sum_{t=1}^{T} \big[x_i \notin U_t\big]\, b_t(x_i) \Big) \neq y_i \right] \to \min$$

Это несмещённая оценка обобщающей способности.

Breiman L. Random Forests // Machine Learning. 2001.

Резюме

- RF — один из самых сильных методов машинного обучения.
- RF обычно лишь немного уступает градиентному бустингу.
- Бэггинг позволяет вычислять оценки out-of-bag:
  — для оптимизации числа базовых алгоритмов T,
  — для оценивания важности признаков,
  — для выбора параметра k в RF.
- RSM хорош, когда много неинформативных признаков.
- Бэггинг, RF, RSM эффективно распараллеливаются.

Линейные композиции для классификации и регрессии

Задача восстановления зависимости $y\colon X \to Y$ по точкам *обучающей выборки* (x_i, y_i), $y_i = y(x_i)$, $i = 1, \ldots, \ell$.

Определение

Линейной композицией базовых алгоритмов $a_t(x) = C\big(b_t(x)\big)$, $t = 1, \ldots, T$, называется суперпозиция функций

$$a(x) = C\left(\sum_{t=1}^{T} \alpha_t b_t(x)\right),$$

где $C\colon \mathbb{R} \to Y$ — *решающее правило*, $\alpha_t \geqslant 0$.

- **Пример 1:** классификация на 2 класса, $Y = \{-1, +1\}$;
  $C(b) = \operatorname{sign}(b)$, $a(x) = \operatorname{sign}\big(b(x)\big)$,
  $b\colon X \to \mathbb{R}$ — дискриминантная функция.
- **Пример 2:** регрессия, $Y = \mathbb{R}$,
  $C(b) = b$, $a(x) = b(x)$, решающее правило не используется.

Градиентный бустинг

Линейная композиция базовых алгоритмов:

$$b(x) = \sum_{t=1}^{T} \alpha_t b_t(x), \quad x \in X, \quad \alpha_t \in \mathbb{R}_+.$$

Функционал качества с произвольной функцией потерь $\mathscr{L}(b, y)$:

$$Q(\alpha, b) = \sum_{i=1}^{\ell} \mathscr{L}\Bigg(\underbrace{\underbrace{\sum_{t=1}^{T-1} \alpha_t b_t(x_i)}_{u_{T-1,i}} + \alpha b(x_i)}_{u_{T,i}}, y_i\Bigg) \to \min_{\alpha, b}.$$

Ищем вектор $u = \big(b(x_i)\big)_{i=1}^{\ell}$ из R^{ℓ}, минимизирующий $Q(\alpha, b)$.
$u_{T-1} = \big(u_{T-1,i}\big)_{i=1}^{\ell}$ — текущее приближение вектора u
$u_T = \big(u_{T,i}\big)_{i=1}^{\ell}$ — следующее приближение вектора u

---

Friedman G. Greedy Function Approximation: A Gradient Boosting Machine. 1999.

Параметрическая аппроксимация градиентного шага

- Градиентный метод минимизации $Q(u) \to \min, \quad u \in \mathbb{R}^\ell$:

$$u_0 := \text{начальное приближение};$$
$$u_{T,i} := u_{T-1,i} - \alpha g_i, \quad i = 1, \ldots, \ell;$$

$g_i = \mathscr{L}'\big(u_{T-1,i},\, y_i\big)$ — компоненты вектора градиента,
α — градиентный шаг.

- Добавление базового алгоритма b_T:

$$u_{T,i} := u_{T-1,i} + \alpha b_T(x_i), \quad i = 1, \ldots, \ell$$

Будем искать такой базовый алгоритм b_T, чтобы вектор $\big(b_T(x_i)\big)_{i=1}^\ell$ приближал вектор антиградиента $\big(-g_i\big)_{i=1}^\ell$:

$$b_T := \arg\max_b \sum_{i=1}^\ell \big(b(x_i) + g_i\big)^2$$

Алгоритм градиентного бустинга (Gradient Boosting)

Вход: обучающая выборка X^ℓ; параметр T;
Выход: базовые алгоритмы и их веса $\alpha_t b_t$, $t = 1, \ldots, T$;

1: инициализация: $u_i := 0$, $i = 1, \ldots, \ell$;
2: **для всех** $t = 1, \ldots, T$
3: найти базовый алгоритм, приближающий градиент:
$$b_t := \arg\min_b \sum_{i=1}^{\ell} \big(b(x_i) + \mathscr{L}'(u_i, y_i)\big)^2;$$
4: решить задачу одномерной минимизации:
$$\alpha_t := \arg\min_{\alpha>0} \sum_{i=1}^{\ell} \mathscr{L}\big(u_i + \alpha b_t(x_i), y_i\big);$$
5: обновить значения композиции на объектах выборки:
$u_i := u_i + \alpha_t b_t(x_i)$; $i = 1, \ldots, \ell$;

Стохастический градиентный бустинг (SGB)

Известно, что рандомизации могут повышать качество композиции за счёт повышения различности базовых алгоритмов (на этом основаны bagging, RF, RSM)

Идея:
на шагах 3–5 использовать не всю выборку X^ℓ,
а случайную подвыборку с повторениями, как в бэггинге.

Преимущества:

- улучшается качество
- улучшается сходимость
- уменьшается время обучения

Friedman G. Stochastic Gradient Boosting. 1999.

Частный случай — алгоритм AdaBoost

Исторически первый вариант бустинга (1995).
Задача классификации на два класса, $Y = \{-1, +1\}$,

$\mathscr{L}(b(x_i), y_i) = e^{-b(x_i)y_i}$ — экспоненциальная функция потерь, убывающая функция отступа $M_i = b(x_i)y_i$

Преимущества:

- для обучения b_t на каждом шаге t решается стандартная задача минимизации взвешенного эмпирического риска
- задача оптимизации α_t решается аналитически

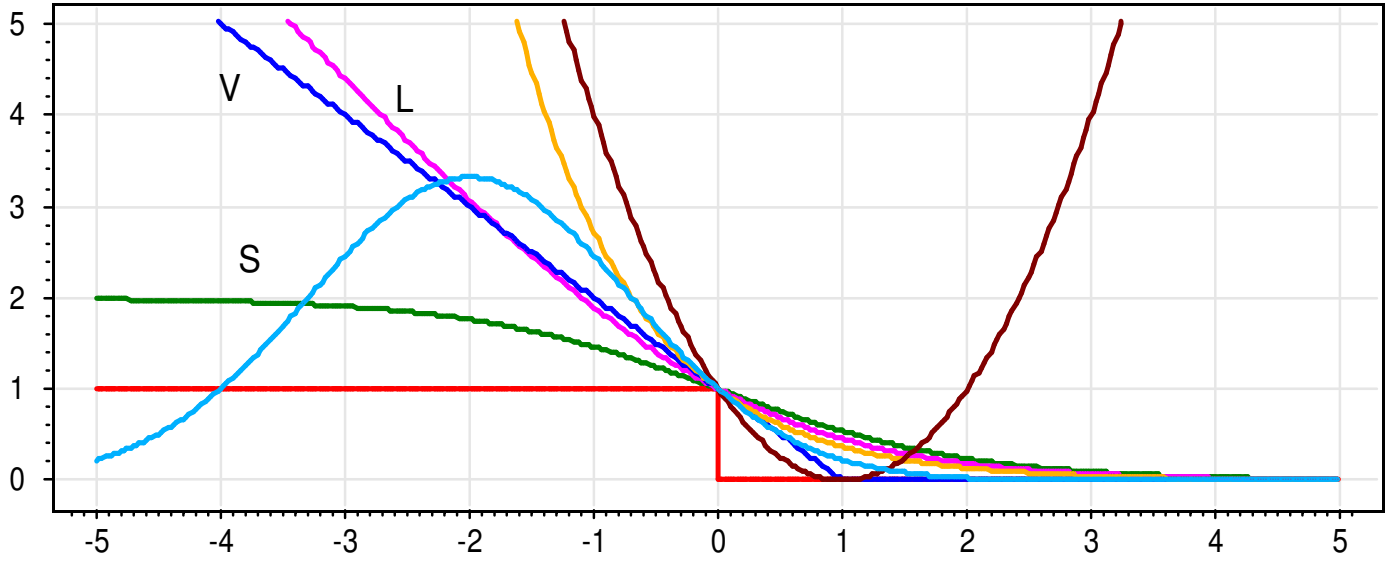
Недостаток:

- AdaBoost слишком чувствителен к выбросам из-за экспоненциального роста функции потерь при $M_i < 0$

Freund Y., Schapire R. E. A decision-theoretic generalization of on-line learning and an application to boosting. 1995.

Частные случаи при различных функциях потерь $\mathscr{L}$

Функции потерь $\mathscr{L}(M)$ в задачах классификации на два класса



Градиентный бустинг над деревьями

Решающее дерево — это кусочно-постоянная функция:

$$b(x) = \sum_{t=1}^{T} \alpha_t [x \in \Omega_t],$$

где T — число листьев,
Ω_t — область t-го листа,
α_t — прогноз в t-м листе.

Идея: каждый лист — базовый алгоритм $b_t(x) = [x \in \Omega_t]$; градиентным шагом определяется прогноз α_t в t-м листе:

$$\alpha_t = \arg\min_{\alpha > 0} \underbrace{\sum_{x_i \in \Omega_t} \mathscr{L}\left(u_{t-1,i} + \alpha, y_i\right)}_{\text{суммарная потеря в } t\text{-м листе}}.$$

После определения всех α_t можно добавить в композицию следующее дерево, оптимизировав его структуру по MSE.

Градиентный бустинг над деревьями

Оптимизация прогнозов в листьях:

$$\alpha_t = \arg\min_{\alpha>0} \sum_{x_i \in \Omega_t} \mathscr{L}\left(u_{t-1,i} + \alpha, y_i\right).$$

Для некоторых функций потерь решение находится аналитически:

- средний квадрат ошибок, MSE, $\mathscr{L}(b, y) = (b - y)^2$:

$$\alpha_t = \frac{1}{|\Omega_t|} \sum_{x_i \in \Omega_t} \left(y_i - u_{t-1,i}\right).$$

- средняя абсолютная ошибка, MAE, $\mathscr{L}(b, y) = |b - y|$:

$$\alpha_t = \operatorname*{median}_{x_i \in \Omega_t} \left\{y_i - u_{t-1,i}\right\}.$$

В общем случае аналитического решения нет.

Резюме

- Градиентный бустинг — наиболее общий из всех бустингов:
  — произвольная функция потерь
  — произвольное пространство оценок R
  — подходит для регрессии, классификации, ранжирования
- Важное открытие середины 90-х: обобщающая способность бустинга не ухудшается с ростом сложности T
- Стохастический вариант SGB — лучше и быстрее
- Градиентный бустинг над решающими деревьями часто работает лучше, чем случайный лес
- Технология Yandex.MatrixNet — это градиентный бустинг над «небрежными» решающими деревьями ODT (ODT — oblivious decision tree)